Lab 2 - Robot Model: URDF, xacro, TF and RViz

Aim. Describe a differential-drive robot (**DiffBot**) in a URDF file written with xacro, publish it with `robot_state_publisher`, see it in RViz2 and read its TF tree.

Time. 45 min. **Needs.** Only this folder (no compiler: the package is pure Python).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (`winget install --id prefix-dev.pixi -e --source winget`), and nothing else.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

- A **URDF** is an XML description: **links** (rigid bodies, with shape, colour, mass) joined by **joints** (parent -> child, with an origin offset and a type). Joint types used here: `fixed` (never moves) and `continuous` (rotates for ever - the wheels).
- **xacro** adds constants (`xacro:property`), arithmetic (`${wheel_separation / 2}`) and macros (`xacro:macro`) so the file has no repeated numbers. DiffBot defines `wheel_radius = 0.05 m`, `wheel_separation = 0.30 m` once; a robot simulator or odometry node written for this robot must use the same numbers (the unit test below checks the wheel position).
- **TF** is ROS's tree of coordinate frames. `robot_state_publisher` reads the URDF from the parameter `robot_description` and the wheel angles from `/joint_states`, and publishes one frame per link: fixed joints on `/tf_static` (once), moving joints on `/tf` (continuously).

```
base_footprint --fixed (z = +0.05, one wheel radius)--> base_link --continuous--> left_wheel_link
(y = +0.15)
                                                                  --continuous--> right_wheel_link
(y = -0.15)
                                                                  --fixed-------> caster_link,
lidar_link (x = +0.08, z = +0.07)
```

Procedure

1. Expand and check the model. xacro turns the macro file into plain URDF; `check_urdf` parses it and prints the link tree.

```
pixi run build-core
pixi run xacro ws/src/diffbot_description/urdf/diffbot.urdf.xacro -o diffbot.urdf
pixi run check_urdf diffbot.urdf
Remove-Item diffbot.urdf
```

2. T1 - show the robot. RViz2 and a *Joint State Publisher* window with two sliders open. You should see a blue box with two black wheels, a grey caster and a red lidar disc. Drag the `left_wheel_joint` slider: the left wheel spins.

```
pixi run demo-model
```

3. T2 - what is being published.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
pixi run ros topic echo /joint_states --no-daemon --once --spin-time 5
```

4. T2 - read a transform. `tf2_echo <parent> <child>` prints where the child frame is relative to the parent. Stop it with Ctrl+C. First a fixed joint, then a wheel joint (drag the slider while it runs: the rotation changes, the translation does not).

```
pixi run ros run tf2_ros tf2_echo base_footprint lidar_link
```

```
pixi run ros run tf2_ros tf2_echo base_link left_wheel_link
```

The lidar is at z = 0.12 m = 0.05 (wheel radius) + 0.05 (half the body height) + 0.02. The left wheel is 0.15 m to the left (= separation / 2).

5. T2 - draw the whole tree. Writes `frames_<time>.pdf` into the current folder (open it; delete it afterwards).

```
pixi run ros run tf2_tools view_frames
Remove-Item frames_*
```

6. Break it on purpose. Widen the robot: change `wheel_separation` from 0.30 to 0.40 in the xacro. The unit test that pins the wheel positions (+-0.15 m) notices:

```
(Get-Content ws\src\diffbot_description\urdf\diffbot.urdf.xacro) -replace 'name="wheel_separation"
value="0.30"','name="wheel_separation" value="0.40"' | Set-Content
ws\src\diffbot_description\urdf\diffbot.urdf.xacro
pixi run test-unit
```

Undo it:

```
(Get-Content ws\src\diffbot_description\urdf\diffbot.urdf.xacro) -replace 'name="wheel_separation"
value="0.40"','name="wheel_separation" value="0.30"' | Set-Content
ws\src\diffbot_description\urdf\diffbot.urdf.xacro
pixi run test-unit
```

Observations (recorded)

Quantity	Value
Links / joints	6 links (`base_footprint` , `base_link` , 2 wheels, caster, lidar) and 5 joints
`base_footprint` -> `lidar_link`	(0.080, 0.000, 0.120)
`base_link` -> `left_wheel_link`	(0.000, 0.150, 0.000)
`/joint_states` names	`left_wheel_joint` , `right_wheel_joint` (only the *continuous* joints need a value)

Verification

```
pixi run test-class2
```

Checks: `/robot_description` is published and names `diffbot` ; the fixed TF `base_footprint -> lidar_link` resolves; after joint states arrive, `base_link -> left_wheel_link` has y = 0.15 m; RViz2 stays alive for 25 s. (The RViz2 part runs without a screen, so it proves only that the launch does not crash. The picture in step 2 is the real check.)

Questions

1. What would change in the TF tree if `lidar_joint` were `continuous` instead of `fixed` ? Which topic would you then have to feed?
2. Why is `base_footprint` the root and `base_link` raised by one wheel radius?
3. Add a camera link 0.12 m in front of the lidar. Which unit test (if any) fails, and why or why not? (Read `ws\src\diffbot_description\test\test_urdf.py` .)
4. Which value in the xacro would you change to make the robot wider, and what else in the course must then change?

If it does not work

Symptom	Fix
RViz shows nothing / grey	set *Fixed Frame* to `base_footprint` ; wait for the model to load
`error creating render window`	you are in offscreen mode (`QT_QPA_PLATFORM`); open a normal PowerShell

Symptom	Fix
`tf2_echo` waits for ever	`demo-model` is not running